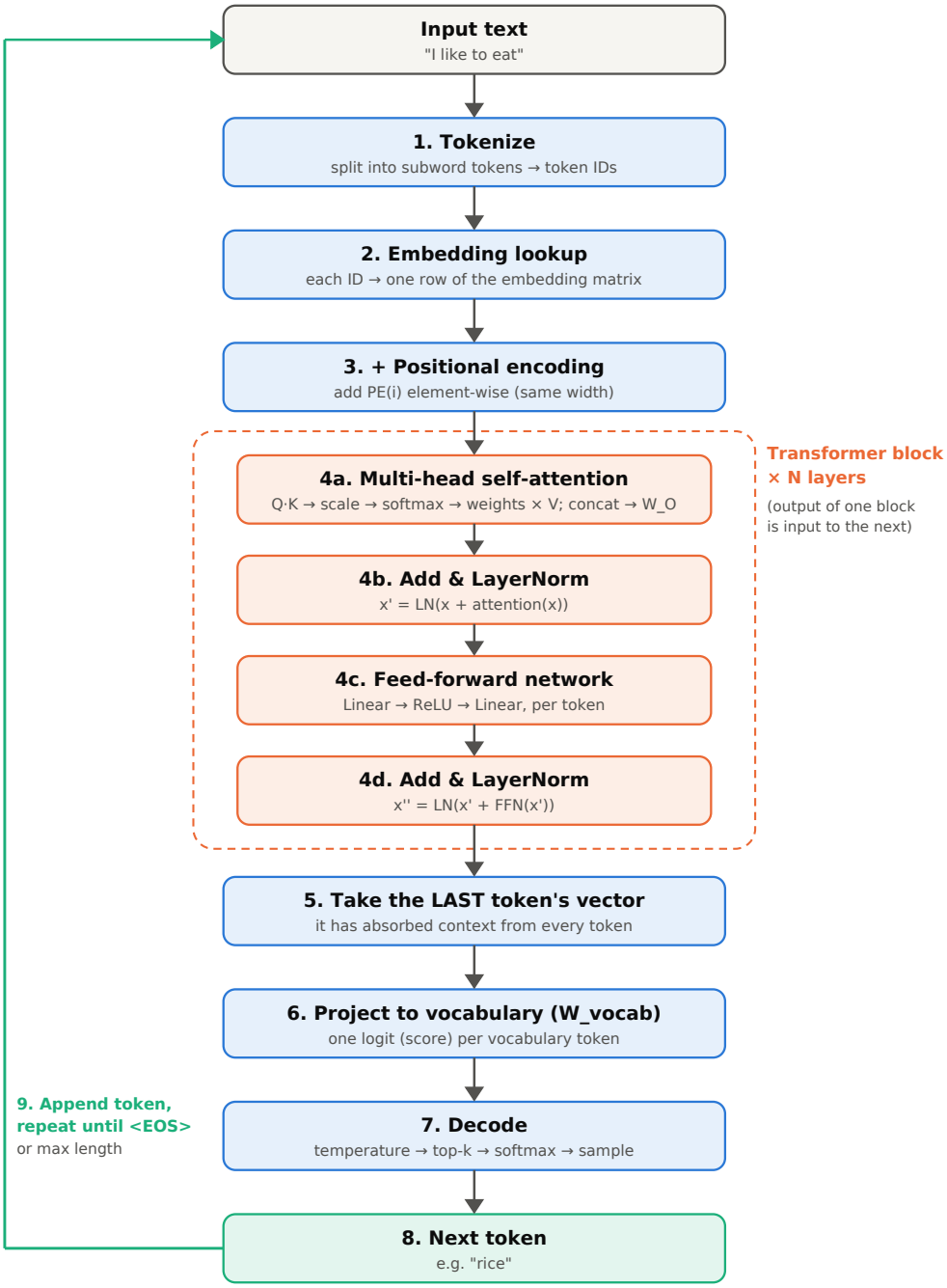
How a Transformer Generates Text

AI Engineer Roadmap · Phase 2 · Topic 8 — Bridge to Transformers · study notes for Mario

Part 1 — The big picture

A large language model does exactly one thing, over and over: given the text so far, it predicts the single next token. Writing a whole answer is just that one prediction repeated in a loop, with each new token glued onto the end of the input before the next round. Everything else in the architecture exists to make that one prediction as good as possible.



Blue boxes turn text into vectors and vectors back into a choice; the dashed orange group is repeated N times (e.g. 12, 32, 96 layers); the green loop is what makes text appear token by token.

The pipeline has three phases. First, text is turned into numbers the network can work with: it is split into tokens, each token is looked up in an embedding table to get a vector, and a positional encoding is added so the model knows word order. Second, those vectors are passed through a stack of identical transformer blocks. Inside each block, attention lets every token gather information from every other token, and a small feed-forward network then refines each token on its own. After many blocks, each token's vector is a rich, context-aware description of what it means in this sentence. Third, the last token's vector is scored against every word in the vocabulary, the scores are turned into probabilities, and one token is picked. That token is appended and the whole thing runs again.

The pipeline in one table

Stage	What goes in	What comes out	Why it exists
Tokenize	Raw text	List of token IDs	Networks need numbers; subwords handle any word
Embed	Token IDs	One vector per token	Gives each token a learned meaning
Positional encoding	Token vectors	Same-size vectors, now order-aware	Attention alone ignores word order
Attention (+ W_O)	All token vectors	Each vector enriched with context	Lets tokens read each other
Add & Norm	Input + sub-layer output	Stable, same-size vectors	Keeps info and gradients flowing
Feed-forward	Each token vector alone	Refined vector	Adds non-linear processing
Stack × N	Block output	Deeper abstraction	Composes simple patterns into complex ones
W_vocab	Last token's vector	One logit per vocab token	Turns meaning into word scores
Decode	Logits	One chosen token	Controls greedy vs. creative output
Loop	Chosen token	Longer input	Generates text one token at a time

Three ideas to hold onto

Every token is a vector, and the vector size stays the same all the way through. If the model width is d_model = 512, then the embedding, the positional encoding, every block's output, and the final vector are all 512 numbers long. Blocks change the *contents* of the vectors, not their shape.

All the weights are learned together. The embedding table, every Q/K/V matrix, W_O, the feed-forward weights and W_vocab are all trained at once, by one loss (how wrong was the next-token guess?) and one backward pass. Nothing is trained in a separate stage.

Attention gives global reach from layer 1; depth gives abstraction. Unlike a CNN, a token can look at any other token in the very first block. What stacking adds is composition: later blocks build higher-level meaning out of what earlier blocks found.

Part 2 — The details

1. Tokenization

Text is split into **subword tokens** using a scheme like Byte-Pair Encoding (BPE). Common words are one token ("the"), rarer words split into pieces ("tokenization" → "token" + "ization"), and in the worst case a word falls back to single characters or bytes. That is why the model can talk about a word it has never seen, like "indomie": it is built from smaller pieces that *are* in the vocabulary. Each token maps to an integer ID. Special tokens such as an end-of-sequence marker <EOS> are part of the vocabulary too.

```
"I like to eat"  →  ["I", " like", " to", " eat"]  →  [40, 588, 284, 4483]
```

2. Embeddings

The **embedding matrix** has one row per vocabulary token and d_model columns. Looking up a token means taking its row: that row is the token's embedding vector. The matrix is a learned parameter, so during training, rows for tokens that appeared in the example get nudged by backprop like any other weight. Tokens with similar roles drift toward similar vectors.

```
embedding_matrix: [vocab_size × d_model]
x_i = embedding_matrix[token_id_i]     # one vector, length d_model
```

3. Positional encoding

Attention, as built, treats the input as a bag: shuffle the words and every score and output stays the same. To fix that, a **position vector PE(i)** is **added** (not concatenated) to each embedding. It is the same length as the embedding and covers the full width.

```
PE(pos, 2k)   = sin(pos / 10000^(2k/d_model))
PE(pos, 2k+1) = cos(pos / 10000^(2k/d_model))
x_i' = x_i + PE(i)
```

Each pair of dimensions is a sine/cosine wave at a different speed: early dimensions change fast (fine position, like a clock's second hand), later ones change slowly (coarse position, like the hour hand). Values stay between −1 and 1 and never depend on sentence length, which is what ruled out the raw-index and length-normalized ideas. Two positions could in principle collide on one wave, but colliding on *all* waves at once is practically impossible because many frequencies are used. (Many modern models use learned or rotary position schemes instead, but the purpose is the same.)

4. Self-attention (one head)

Each token produces three vectors from **shared, learned** weight matrices: a **Query** (what am I looking for?), a **Key** (what do I offer?) and a **Value** (what information do I pass on?). The matrices W_Q, W_K, W_V are the same for every token; only the input x differs, so Q, K, V are computed fresh for every token on every forward pass.

```
Q_i = W_Q · x_i     K_i = W_K · x_i     V_i = W_V · x_i
score(i→j) = (Q_i · K_j) / sqrt(d_k)
weights_i  = softmax(score(i→1), ..., score(i→n))
output_i   = Σ_j weights_i[j] · V_j
```

Scores use Q·K; V only enters at the final weighted sum. Dividing by sqrt(d_k) keeps dot products from growing with vector size, which would otherwise make softmax overly peaked. The output for each token is a **vector**, a blend of every token's Value weighted by relevance.

Worked example from our session (1-dimensional Q/K/V to keep the arithmetic small):

Step	hey → hey	hey → world
Q_hey · K_j	0.43 × 0.54 = 0.2322	0.0903
softmax	0.535	0.465
× V_j	0.535 × 0.21	0.465 × 0.175
output_hey	≈ 0.194 (sum of both rows)	

Softmax facts worth remembering. Softmax is shift-invariant: adding the same constant to every score changes nothing, because e^c factors out of the numerator and denominator. Sharpness depends on the *gaps* between scores, not their absolute size: [2, 0] gives 88% / 12%, while [0.23, 0.09] gives 53.5% / 46.5%. Exponentiation handles negative scores cleanly, and base e is used because it is its own derivative, keeping gradients simple.

5. Multi-head attention and W_O

Instead of one attention, the model runs several **heads** in parallel, each with its own W_Q, W_K, W_V and a smaller size (typically d_model / num_heads). Random initialization plus one shared loss pushes heads toward different, complementary patterns (e.g. one tracks the subject, another nearby words). Each head's output has only that head's small size; the heads are then **concatenated** back into one d_model-length vector and multiplied by **W_O**, which mixes the heads' findings together rather than leaving them in separate segments.

```
head_h = Attention(W_Q^h x, W_K^h x, W_V^h x)       # length d_model / H
concat = [head_1 ; head_2 ; ... ; head_H]          # length d_model
attn_out = W_O · concat                              # length d_model
```

6. Residual connection and layer norm (first wrap)

The attention output is **added back** to the block's input, then **layer-normalized** (each token's vector rescaled across its own dimensions to a stable range). The residual means a block only has to learn a *change* to the vector, and it gives gradients a clean path through deep stacks.

```
x' = LayerNorm(x + attn_out)
```

7. Feed-forward network (second wrap)

Attention is mostly weighted averaging, which is linear in the Values. The **feed-forward network** brings back non-linearity: a small two-layer net applied to each token **separately**, with the same weights for every position. The hidden layer is usually about 4× wider than d_model. It does not predict the next word; it just outputs a refined vector, which gets its own residual and layer norm.

```
FFN(x') = W_2 · ReLU(W_1 · x' + b_1) + b_2
x'' = LayerNorm(x' + FFN(x'))      # added to x', not the raw embedding
```

*So layer norm happens **twice per block**, once around each sub-layer. (Many modern models move it before each sub-layer, "pre-norm", but the two-wrap structure is the same.)*

8. Stacking blocks

The output x'' of one block is the input of the next, same shape every time. With N blocks, early layers tend to capture simpler, local relationships and later layers compose them into abstract ones, but every layer can attend across the whole sequence. Depth buys abstraction, not

reach.

9. The output layer

After the last block, take the vector of the **last position**. Multiply it by **W_vocab** (size vocab_size × d_model) to get one **logit** per vocabulary token. This is a plain dot product of the final vector with each token's row, not cosine similarity, so no length normalization happens. Some models tie W_vocab to the embedding matrix. A higher logit means a better fit as the next token.

```
logits = W_vocab · h_last          # length vocab_size
```

10. Decoding: choosing the token

Strategy	How it works	Effect
Greedy	Pick the highest logit	Deterministic, can get repetitive
Sampling	softmax(logits), draw randomly	Varied, sometimes odd
Top-k	Keep only the k best, renormalize, sample	Variety without junk tokens
Temperature T	softmax(logits / T), i.e. (e^score)^(1/T)	T<1 sharpens; T>1 flattens

Your derivation: dividing by T is raising each e^score to the power 1/T. With T < 1 the gaps widen and the top token dominates; as T → 0 it becomes greedy. With T > 1 big scores are damped and unlikely tokens gain probability; as T → ∞ it approaches uniform random.

11. The autoregressive loop

The chosen token is **appended** to the sequence and the whole, longer sequence is run through the pipeline again to predict the next one. This repeats until <EOS> is the chosen token or a maximum length is hit. That is why chat responses stream word by word, and why longer outputs cost more: each new token needs another full pass. (Production systems cache earlier Keys and Values, the "KV cache", so old tokens are not fully recomputed, but logically every step sees the whole sequence.)

12. Training: one loss for everything

Training text supplies its own answers: at every position, the correct output is simply the next token. The model's predicted probabilities are compared against that with **cross-entropy loss**, and one backward pass computes gradients for **every** parameter at once: embeddings, all Q/K/V and W_O matrices, feed-forward weights, layer-norm parameters and W_vocab. It is the same single-cost backprop worked example from Topic 6 (backpropagation), just with far more weights.

> **Corrections from your 24-step synthesis.** (1) PE is the same length as the embedding and is added across the full width, not appended. (2) A single head's output has the head's small size; only after concatenation is it d_model long. (3) LayerNorm runs twice per block, not once. (4) Generation appends the new token and reprocesses the sequence; nothing is "replaced".